

Concepts Learned: Fraud Network Detection Project

A Beginner-to-Advanced Study Guide, with code drawn directly from the project

This guide walks through every concept used while building the fraud detection engine, ordered from foundational to advanced. Each concept includes a plain-English explanation and the actual code you ran in the project, so you can revisit this as a personal reference or study document.

Part 1: Beginner Concepts

1.1 Reading data with pandas

BEGINNER

pandas is Python's core library for working with tabular data (rows and columns, like a spreadsheet). `read_csv()` loads a CSV file into a DataFrame - the main pandas data structure.

```
import pandas as pd
df_train = pd.read_csv("fraudTrain.csv")
df_test = pd.read_csv("fraudTest.csv")
print(df_train.shape, df_test.shape)
print(df_train.columns.tolist())
```

1.2 Inspecting data quality

BEGINNER

Before any modeling, you always check: how big is the data, are there missing values, are there duplicate rows, and what type is each column stored as (number, text, date). Skipping this step is one of the most common beginner mistakes.

```
print(df_train.isnull().sum())      # missing values per column
print(df_train.dtypes)             # data type per column
print(df_train.duplicated().sum()) # fully duplicated rows
print(df_train['amt'].describe())  # count, mean, std, min, quartiles, max
```

1.3 Class imbalance

BEGINNER

In fraud detection (and many real-world problems), the event you care about is rare. Here, only 0.58% of transactions were fraud. This matters because a model that always predicts "not fraud" would score 99.4% accuracy while being completely useless - which is why accuracy alone is a misleading metric for imbalanced problems.

```
print(df_train['is_fraud'].value_counts())
print(df_train['is_fraud'].value_counts(normalize=True) * 100)
# Output: 0 -> 99.42%, 1 -> 0.58%
```

1.4 Datetime handling

BEGINNER

Dates and times arrive from CSV files as plain text (strings). `pd.to_datetime()` converts them into a real datetime object, unlocking date arithmetic (like calculating age) and time-based extraction (hour, day of week, month).

```
df_train['trans_date_trans_time'] = pd.to_datetime(df_train['trans_date_trans_time'])
df_train['dob'] = pd.to_datetime(df_train['dob'])

df_train['age'] = (df_train['trans_date_trans_time'] - df_train['dob']).dt.days // 365
df_train['trans_hour'] = df_train['trans_date_trans_time'].dt.hour
df_train['trans_day'] = df_train['trans_date_trans_time'].dt.dayofweek
df_train['trans_month'] = df_train['trans_date_trans_time'].dt.month
```

1.5 groupby() aggregation

BEGINNER

`groupby()` is one of the most-used pandas operations: it splits data into groups (e.g. by hour, or by category), applies a calculation to each group (mean, count, sum), and returns the result. This is how the fraud-rate-by-hour and fraud-rate-by-category insights were produced.

```
fraud_by_hour = df_train.groupby('trans_hour')['is_fraud'].mean() * 100
fraud_by_cat = df_train.groupby('category')['is_fraud'].mean().sort_values(ascending=False) * 100
print(df_train.groupby('is_fraud')['amt'].describe()) # compare amount stats fraud vs. not
```

1.6 Basic visualization with matplotlib

BEGINNER

`matplotlib` is Python's foundational plotting library. A simple bar chart was enough to reveal that fraud spikes at night - a strong finding from a very short piece of code.

```
import matplotlib.pyplot as plt
fraud_by_hour.plot(kind='bar', color='crimson')
plt.title('Fraud Rate by Hour of Day (%)')
plt.show()
```

Part 2: Intermediate Concepts

2.1 Feature engineering

INTERMEDIATE

Raw columns are rarely the best inputs to a model. Feature engineering means creating new columns that expose patterns more directly. Two engineered features mattered most here:

Binary flag feature - turning a continuous pattern (fraud clusters at certain hours) into a simple 0/1 signal:

```
df_train['is_night'] = df_train['trans_hour'].isin([22,23,0,1,2,3]).astype(int)
```

Z-score / standardization as a feature - measuring how unusual a value is *relative to that entity's own history*, not in absolute terms:

```
customer_amt_stats = df_train.groupby('cc_num')['amt'].agg(['mean', 'std'])
df_train['amt_zscore'] = (df_train['amt'] - df_train['customer_avg_amt']) / df_train['customer_std_amt']
# Result: legit txns average z-score ~0; fraud txns average z-score ~-3.2
```

2.2 Graph theory & network analysis (NetworkX)

INTERMEDIATE

This is the core "network" concept of the project. Instead of treating each transaction as an isolated row, customers and merchants become **nodes**, and each transaction becomes an **edge** connecting them. This is called a **bipartite graph** (two distinct types of nodes: customers and merchants). A MultiGraph is used because the same customer can transact with the same merchant more than once.

```
import networkx as nx
G = nx.from_pandas_edgelist(
    sample, source='cc_num', target='merchant',
    edge_attr=['amt', 'is_fraud', 'trans_hour', 'category'],
    create_using=nx.MultiGraph()
)
print(G.number_of_nodes(), G.number_of_edges())

degrees = dict(G.degree()) # how many connections each node has
top_node = max(degrees, key=degrees.get) # most-connected node
```

Degree is a fundamental graph concept: the number of edges touching a node. A merchant's degree tells you how many transactions (or distinct customers) it's connected to - a proxy for popularity or, in fraud contexts, potential exposure.

2.3 Turning graph structure into ML features

INTERMEDIATE

A graph is useful for visualization, but a model needs numeric columns. "Network features" translate graph structure into per-row numbers: how many distinct merchants a customer touches (degree from the customer's side) and how many distinct customers a merchant serves (degree from the merchant's side).

```
customer_merchant_diversity = df_train.groupby('cc_num')['merchant'].nunique()
merchant_customer_diversity = df_train.groupby('merchant')['cc_num'].nunique()
merchant_fraud_rate = df_train.groupby('merchant')['is_fraud'].mean()
```

2.4 Haversine distance (geospatial feature engineering)

INTERMEDIATE

The Haversine formula calculates the real-world distance between two latitude/longitude points on a sphere (Earth). It's a standard technique whenever you have geographic coordinates and want a distance feature. In this project, it was tested and then correctly ruled out as unhelpful - a good example of how not every plausible feature works, and testing a hypothesis (even a wrong one) is real ML practice.

```
def haversine_distance(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat, dlon = lat2 - lat1, lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
    return 6371 * 2 * np.arcsin(np.sqrt(a)) # 6371 km = Earth's radius
```

2.5 Rolling time windows (velocity features)

INTERMEDIATE

A rolling window looks backward over a fixed span of time (e.g. the last hour) from each row and computes a statistic - here, a count of transactions. This is the concept that makes an engine feel "real-time": at any moment, you can ask "how active has this account been recently?"

```
df_train = df_train.sort_values(['cc_num', 'trans_date_trans_time'])
df_train['trans_count_last_1h'] = (
    df_train.groupby('cc_num')
    .apply(lambda x: x.set_index('trans_date_trans_time')['trans_num'].rolling('1h').count())
    .reset_index(level=0, drop=True).values
)
```

2.6 Train/test split evaluation

INTERMEDIATE

A model is trained on one portion of data (train) and evaluated on a separate, unseen portion (test), so performance numbers reflect genuine generalization rather than memorization. Here the split was time-based (train = earlier period, test = later period) rather than random - a realistic setup for a system that will be deployed forward in time, but one that introduces its own pitfalls (see Section 3.2).

Part 3: Advanced Concepts

3.1 Data leakage

ADVANCED

Data leakage happens when information that would not actually be available at prediction time sneaks into a feature, making a model look better in testing than it will ever perform in reality. A classic example flagged in this project: computing `merchant_fraud_rate` using the entire training set including future transactions relative to any given row. The fix is to only use information that occurred *strictly before* the row being scored.

3.2 Distribution shift from incorrectly-scoped aggregates (the project's core bug)

ADVANCED

This was the single most important debugging lesson in the whole project. Static aggregate features (like a customer's total transaction count) were originally computed *separately* on the train file and the test file. Since the test file only spans a shorter, later period, every customer appeared to have far fewer transactions in test purely because there was less time to count over - not because their real behavior changed. Tree-based models split hard on these count thresholds and were misled by the artificial gap; Logistic Regression's smooth linear weighting was comparatively unaffected. Diagnosing this required directly comparing feature means between train and test:

```
for col in feature_cols:
    print(f"{col}: train_mean={X_train[col].mean():.4f}, test_mean={X_test[col].mean():.4f}")
# customer_txn_count: train=1818.7 test=781.5 <- huge, suspicious gap
```

3.3 Causal / expanding-window feature computation

ADVANCED

The fix for both the leakage and the distribution-shift bug at once: concatenate train and test into one chronological timeline, compute every aggregate using an **expanding window** (a running calculation that only looks backward) or a running count, then split back into train/test using a saved tag column. `expanding().mean().shift(1)` means "the average of everything before this row, not including this row itself" - the `shift(1)` is what prevents the current row's own outcome from leaking into its own feature.

```
df_all['is_train'] = ... # tag before combining
df_all = pd.concat([df_train, df_test]).sort_values(['cc_num', 'trans_date_trans_time'])

df_all['customer_txn_count'] = df_all.groupby('cc_num').cumcount() + 1

df_all['customer_avg_amt'] = (
    df_all.groupby('cc_num')['amt']
    .transform(lambda x: x.expanding().mean().shift(1))
)
df_all['merchant_fraud_rate'] = (
    df_all.groupby('merchant')['is_fraud']
    .transform(lambda x: x.expanding().mean().shift(1))
)

df_train = df_all[df_all['is_train']==1].drop(columns=['is_train'])
df_test = df_all[df_all['is_train']==0].drop(columns=['is_train'])
```

Impact: this single fix raised Random Forest's AUC-PR from 0.043 to 0.693 - a more than 10x improvement, proving the original poor performance was a data problem, not a model problem.

3.4 Handling class imbalance in model training

ADVANCED

Two different mechanisms were used to tell models to pay more attention to the rare fraud class: scikit-learn's `class_weight='balanced'` automatically reweights classes inversely to their frequency; XGBoost's `scale_pos_weight` explicitly sets how many times more a positive (fraud) example should count versus a negative one. An advanced lesson learned: more aggressive reweighting is not always better - in this project, moderate/full weighting outperformed `weight=1`, but the relationship between weight and AUC-PR had to be tested empirically, not assumed.

```
LogisticRegression(class_weight='balanced', max_iter=1000)
RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42, n_jobs=-1)

scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum() # ratio of negatives to positives
XGBClassifier(scale_pos_weight=scale_pos_weight, eval_metric='aucpr', ...)
```

3.5 Choosing the right evaluation metric

ADVANCED

For a 99.4%-vs-0.6% imbalanced problem, plain accuracy is meaningless. The project used:

Metric	What it measures
Precision	Of everything flagged as fraud, what fraction actually was fraud
Recall	Of all real fraud, what fraction did the model catch
F1-score	Harmonic mean of precision and recall - a single balance metric
ROC-AUC	Overall ability to rank fraud above non-fraud, less sensitive to imbalance
AUC-PR	Precision-recall tradeoff curve area - the most informative metric for rare-event detection

```
from sklearn.metrics import classification_report, roc_auc_score, average_precision_score
print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
print("AUC-PR:", average_precision_score(y_test, y_proba))
```

3.6 Decision threshold tuning

ADVANCED

A classifier outputs a probability, not a verdict - by default, 0.5 is treated as the cutoff for "fraud", but that default is rarely optimal for imbalanced problems. Plotting precision and recall across many thresholds lets you pick an operating point matching real business priorities (catch more fraud vs. reduce false alarms).

```
from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_test, y_proba_rf)
for t in [0.1, 0.2, 0.3, 0.4, 0.5]:
    idx = np.argmin(np.abs(thresholds - t))
    print(f"Threshold={t}: Precision={precisions[idx]:.4f}, Recall={recalls[idx]:.4f}")

y_pred_final = (y_proba_rf >= 0.2).astype(int) # chosen operating threshold
```

3.7 Model comparison: linear vs. ensemble vs. boosting

ADVANCED

Three model families were compared, each representing a different modeling philosophy: **Logistic Regression** (a single linear decision boundary - simple, fast, robust to noisy features, but limited in capturing non-linear interactions); **Random Forest** (an ensemble of many decision trees trained on random subsets of data/features, then averaged - captures non-linear patterns and feature interactions); **XGBoost** (gradient boosting - trees are built sequentially, each correcting the errors of the previous ones, generally the strongest performer on structured/tabular data). A key lesson: the "best" algorithm on paper can still lose to a simpler one if the input features are flawed - algorithm choice matters less than data quality.

3.8 Feature importance interpretation

ADVANCED

Tree-based models can report which features contributed most to their decisions, based on how much each feature reduced impurity across all splits in all trees. This was used twice in the project: first to discover the model was over-relying on just two raw features (a sign the feature set was too shallow), and later to confirm the engineered z-score and network features were pulling real, meaningful weight.

```
importances = pd.Series(rf_model.feature_importances_, index=feature_cols).sort_values(ascending=False)
print(importances)
```

3.9 Graph visualization for pattern discovery

ADVANCED

Beyond using graph structure as numeric features, the graph itself was visualized using a spring layout (a physics-based algorithm that pulls connected nodes together and pushes unconnected nodes apart), with fraud edges specifically colored to visually separate suspicious connections from normal traffic - a technique real fraud analysts use to spot clusters worth investigating.

```
pos = nx.spring_layout(G_sub, k=0.3, seed=42, iterations=20)
nx.draw_networkx_edges(G_sub, pos, edgelist=fraud_edges, edge_color='red', width=2, alpha=0.8)
nx.draw_networkx_nodes(G_sub, pos, nodelist=customer_nodes, node_color='steelblue', node_size=30)
nx.draw_networkx_nodes(G_sub, pos, nodelist=merchant_nodes, node_color='orange', node_size=80)
```

Summary: Concept Progression

Level	Concepts
Beginner	pandas basics, data quality checks, class imbalance awareness, datetime handling, groupby aggregation, ba
Intermediate	feature engineering, graph theory & NetworkX, graph-to-feature translation, Haversine distance, rolling time v
Advanced	data leakage, distribution shift from mis-scoped aggregates, causal/expanding-window features, class-imbala

The most important overall takeaway: in real-world machine learning, especially with time-ordered data, **how features are computed matters more than which algorithm you choose**. The biggest performance jump in this entire project (AUC-PR 0.043 to 0.693) came from fixing how features were calculated, not from switching models.